MLPerf Inference Benchmark

论文精读笔记

arXiv:1911.02549v2

基于 Vijay Janapa Reddi 等人论文的详细解读

March 5, 2026

Contents

1 摘要 (Abstract) **3**

2 引言 (Introduction) **3**
2.1 五大贡献 5

3 推理基准测试的挑战 (Inference-Benchmarking Challenges) **7**
3.1 模型的多样性 7
3.2 部署场景的多样性 9
3.3 推理系统的多样性 9

4 基准测试设计 (Benchmark Design) **11**
4.1 代表性、广泛可访问的工作负载 12
4.2 稳健的质量目标 13
4.3 现实的端用户场景 14
4.4 统计置信的尾延迟边界 16

5 推理提交系统 (Inference Submission System) **19**
5.1 被测系统 20
5.2 负载生成器（LoadGen） 21
5.3 数据集 22
5.4 准确度检查器 23

6 提交系统评估 (Submission-System Evaluation) **23**
6.1 结果提交、分区和类别 23
6.2 结果审查 24
6.3 结果报告 24

7 基准测试评估 (Benchmark Assessment) **25**
7.1 任务覆盖 25
7.2 场景使用 26
7.3 处理器类型和软件框架 28
7.4 系统多样性 29
7.5 开放分区 31

8 经验教训 (Lessons Learned) 31
8.1 模型：广度 vs 用例深度 31
8.2 度量指标：延迟 vs 吞吐量 31
8.3 数据集：公共 vs 私有 31
8.4 性能：建模 vs 实测 32
8.5 流程：审计和可审计性 32

9 AI/ML 基准测试的先前工作 (Prior Art in AI/ML Benchmarking) 32

10 结论 (Conclusion) 34

附录：参考文献一览表 36

1 摘要 (Abstract)

原文翻译

机器学习（ML）硬件和软件系统的需求正在蓬勃增长。在 ML 应用的推动下，各种 ML 推理系统的数量急剧增长。超过 100 个组织正在构建 ML 推理芯片，而采用现有模型的系统在功耗上至少跨越三个数量级，在性能上跨越五个数量级；它们的范围从嵌入式设备到数据中心解决方案。驱动硬件的是十几种甚至更多的软件框架和库。ML 硬件和 ML 软件的无数组合使得以架构中立、代表性和可重现的方式评估 ML 系统性能变得极具挑战性。业界迫切需要行业范围的标准 ML 基准测试和评估标准。MLPerf Inference 正是应此需求而生。

在本文中，我们介绍了我们用于评估 ML 推理系统的基准测试方法。由 30 多个组织以及超过 200 名 ML 工程师和从业者共同推动，MLPerf 规定了一套规则和最佳实践，以确保在架构差异巨大的系统之间实现可比较性。首次征集提交结果便获得了来自 14 个组织的超过 600 个可重现的推理性能测量结果，代表了超过 30 个系统，展示了广泛的能力范围。这些提交结果证明了该基准测试的灵活性和适应性。

通俗解释

简单来说，这篇论文介绍的是一个叫做 **MLPerf Inference** 的"考试标准"。

想象一下：现在有上百家公司在做 AI 芯片，每家公司都说自己的芯片最快、最好。但问题是——大家用的测试方法都不一样，就像不同学校的考试题目不同，无法直接比较谁更优秀。

MLPerf Inference 就像是一套"全国统考"：它规定了考什么题目（哪些 AI 任务）、怎么考（什么场景）、及格线是多少（质量要求），这样不同公司的芯片就可以公平比较了。

这个标准由 30 多个组织、200 多位工程师共同制定，首次测试就收到了 14 个组织提交的 600 多个测试结果，涵盖了从手机芯片到数据中心服务器的各种系统。

关键概念

- **核心问题**：ML 推理系统种类繁多（100+ 家芯片公司），缺乏统一评估标准
- **解决方案**：MLPerf Inference——业界标准的 ML 推理基准测试
- **参与规模**：30+ 组织，200+ 工程师
- **首轮成果**：14 个组织，600+ 提交结果，30+ 系统
- **性能跨度**：功耗跨 3 个数量级，性能跨 5 个数量级

2 引言 (Introduction)

原文翻译

机器学习（ML）为从计算机视觉**【引用说明：He2016 ResNet、Goodfellow2014 GAN、Liu2016 SSD、Krizhevsky2012 AlexNet——这些是计算机视觉领域的里程碑工作，说明 ML 应用范围广泛】**和自然语言处理**【引用说明：Vaswani2017 Transformer Attention 机制、Devlin2018 BERT——NLP 领域的代表性工作】**到自动驾驶汽车**【引用说明：Xu2018 PointFusion 3D检测、Badrinarayanan2017 SegNet**

语义分割——自动驾驶中的关键ML应用】和自主机器人**【引用说明：Levine2018 机器人抓取学习——展示 ML 在机器人领域的应用】**的各种应用提供动力。虽然 ML 模型训练一直是开发瓶颈和一项相当大的开销**【引用说明：Amodei2018 AI与算力——OpenAI关于训练算力需求指数增长的分析】**，但推理已成为一种关键工作负载。模型每天可以服务多达 200 万亿次查询，并执行超过 60 亿次翻译**【引用说明：Lee2019 Facebook基础设施加速——展示了推理规模之大】**。为了满足这些日益增长的计算需求，硬件、软件和系统开发者已针对各种用例的推理性能进行了优化，设计了优化的 ML 硬件和软件。据估计，超过 100 家公司正瞄准专用推理芯片**【引用说明：Kanter2019 超算大会报告——统计了AI芯片公司数量】**。相比之下，仅约 20 家公司瞄准训练芯片**【引用说明：Tang2019 AI-Chip列表——AI芯片公司汇总】**。

通俗解释

这段开头说明了 ML（机器学习）推理的重要性：
训练 vs 推理的关系可以类比为"编写教科书 vs 学生做题"：

- **训练**：像写教科书，耗时但只需做一次（或很少次）
- **推理**：像学生做题，每次有新输入就要"做一次题"。Facebook 每天要做 200 万亿次推理！

所以虽然训练很贵，但推理因为量大而成为更关键的工作负载。这也是为什么 100+ 家公司做推理芯片，但只有 20 家做训练芯片。

原文翻译

每个 ML 系统采用独特的推理方法，在延迟、吞吐量、功耗和模型质量之间进行权衡。结果是 ML 任务、模型、数据集、框架、工具集、库、架构和推理引擎的众多可能组合，使得评估推理性能几乎成为不可能的任务。任务范围广泛，包括但不限于图像分类和定位、目标检测和分割、机器翻译、自动语音识别、文本转语音和推荐系统。即使对于特定任务（如图像分类），也有许多可行的 ML 模型。这些模型服务于各种场景，从在智能手机上拍摄单张照片到在自动驾驶汽车中通过多个摄像头持续并发检测行人。因此，ML 任务具有截然不同的质量要求和实时处理需求。即使模型的函数和操作的实现也可能高度依赖特定框架，它们增加了设计任务的复杂性。要量化这些权衡，ML 领域需要一个架构中立、具有代表性且可重现的基准测试。

通俗解释

这段解释了为什么基准测试这么难做。想象一个"排列组合爆炸"的问题：
假设你要评估一个 AI 系统，你需要选择：

- 什么任务？（图像分类、目标检测、翻译.....10+ 种）
- 什么模型？（ResNet、MobileNet、BERT.....100+ 种）
- 什么框架？（TensorFlow、PyTorch.....10+ 种）
- 什么硬件？（CPU、GPU、ASIC.....各种）
- 什么场景？（手机单张照片 vs 自动驾驶多路摄像头）

这些组合起来就是天文数字！而且不同场景对"好"的定义完全不同——手机上追求低延迟，服务器上追求高吞吐。所以需要一个公平、全面的基准测试。

原文翻译

学术和工业组织都开发了 ML 推理基准测试。先前工作的例子包括来自工业界的 AIMatrix**【引用说明：Alibaba的AI加速器基准测试】**、EEMBC MLMark**【引用说明：嵌入式微处理器基准联盟的ML推理基准】**和 AIXPRT**【引用说明：Principled Technologies开发的封闭AI基准测试】**，以及来自学术界的 AI Benchmark**【引用说明：Ignatov2019 首个移动推理基准】**、TBD**【引用说明：Zhu2018 多伦多大学的训练基准】**、Fathom**【引用说明：Adolf2016 哈佛大学的深度学习参考工作负载集】**和 DAWNBench**【引用说明：Coleman2017 Stanford的端到端深度学习基准竞赛，是MLPerf的前身之一】**。每个基准测试都对 ML 基准测试做出了实质性贡献，但它们是在没有来自 ML 系统设计者的全行业输入的情况下开发的。因此，在代表性的机器学习模型、度量指标、任务和规则上没有达成共识。

在本文中，我们介绍了 MLPerf Inference，一个标准的 ML 推理基准测试套件，具有适当的度量指标和基准测试方法（补充了 MLPerf Training**【引用说明：Mattson2019 MLPerf Training——MLPerf的训练基准，本文的推理基准与之配套】**），用于公平测量 ML 硬件、软件和服务的推理性能。工业界和学术界使用来自 30 多个组织的研究人员和开发人员的输入共同开发了基准测试套件及其方法。

超过 200 名 ML 工程师和从业者为此付出了努力。MLPerf Inference 是由架构、系统和机器学习专家共同锻造的关于最佳基准测试技术的共识。我们解释了为什么设计正确的 ML 基准测试度量指标、创建现实的 ML 推理场景以及标准化评估方法能够实现推理质量的现实性能优化。

2.1 五大贡献

原文翻译

① **我们选择了具有代表性的工作负载以确保可重现性和可访问性**。ML 生态系统中模型众多。在这些模型之间比较和对比 ML 系统性能是非常困难的，因为它们在模型复杂度和执行特性方面差异巨大。此外，像 ``ResNet-50'' 这样的名称无法唯一或可移植地描述一个模型。因此，用不稳定的基线来量化系统性能改进是困难的。

MLPerf 的一个主要贡献是选择了允许可重现测量的代表性模型。基于行业共识，MLPerf Inference 包含成熟且获得社区支持的模型。因为行业已经研究了它们并能构建高效系统，基准测试是可访问的，并提供了 ML 系统技术的快照。MLPerf 模型也是开源的，使它们成为潜在的研究工具。

原文翻译

② **我们确定了用于现实评估的场景**。ML 推理系统的范围从深度嵌入式设备到智能手机再到数据中心。它们有各种现实世界的应用和许多优值指标（figures of merit），每个都需要多个性能度量指标。正确的度量指标反映生产用例，不仅允许 MLPerf 而且允许出版物展示实际 ML 系统的性能。

MLPerf Inference 由四个评估场景组成：**单流**（single-stream）、**多流**（multistream）、**服务器**（server）和**离线**（offline）。我们通过调查 MLPerf 的广泛成员（包括客户和供应商）来确定它们。这些场景代表了许多关键推理应用。我们展示了性能在这些场景及其对应度量指标下可以发生巨大变化。MLPerf Inference 提供了一种模拟被测推理系统现实行为的方法；这种特性在 AI 基准测试中是独一无二的。

原文翻译

③ **我们根据现实世界用例规定了目标质量和尾延迟边界**。质量和性能对于所有形式的 ML 都是密切相关的。系统架构有时会牺牲模型质量来降低延迟、降低总拥有成本（TCO）或提高吞吐量。准确度、延迟和 TCO 之间的权衡是特定于应用的。用 1% 的模型准确度换取 50% 更低的 TCO，在识别猫咪照片时是明智的，但在自动驾驶汽车检测行人时则是有风险的。
为了反映真实部署的这一方面，MLPerf 定义了模型质量目标。我们为每个模型和场景建立了推理延迟和模型质量的目标。延迟边界和目标质量基于从 ML 系统最终用户和 ML 从业者收集的输入。随着 MLPerf 根据行业需求改进这些参数，更广泛的研究社区可以跟踪它们以保持相关性。

原文翻译

④ **我们设置了宽松的规则，允许用户展示硬件和软件能力**。社区已经接受了各种语言和库，因此 MLPerf Inference 是一个语义级别的基准测试。我们指定任务和规则，但将实现留给提交者。
我们的基准测试允许提交者优化参考模型，通过其首选软件工具链运行它们，并在其选择的硬件上执行。因此，MLPerf Inference 有两个分区：**封闭**（closed）和**开放**（open）。严格的规则管理封闭分区，该分区解决了缺乏标准推理基准测试工作流程的问题。另一方面，开放分区允许提交者更改模型并展示不同的性能和质量目标。

原文翻译

⑤ **我们提出了一种推理基准测试方法，允许模型频繁更改同时保留上述贡献**。ML 发展迅速，因此任何基准测试的挑战不是执行任务本身，而是实现一种能够经受快速变化的方法。
MLPerf Inference 高度关注基准测试的模块化设计，使添加新模型和任务的成本更低，同时保留使用场景、目标质量和基础设施。正如我们在第 7.5 节中所展示的，我们的设计允许用户轻松添加新模型。我们计划扩展范围以包含更多领域和任务。我们正在努力添加新模型（如推荐系统和时间序列）、新场景（如"突发"模式）、更好的工具（如移动应用程序）和更好的度量指标（如计时预处理），以更好地反映整个 ML 管道的性能。

通俗解释

论文的五大贡献可以用"建立一场公平考试"来类比：

1. **选代表性的"考题"**：不是随便出题，而是选大家都认可的、有意义的模型（如 ResNet-50）
2. **设计现实的"考试情境"**：四种场景模拟真实使用——手机单次拍照（单流）、自动驾驶多摄像头（多流）、在线翻译服务器（服务器）、批量处理相册（离线）
3. **设定"及格线"**：不仅要快，还要准确。识别猫可以稍微不准，但自动驾驶识别行人必须高度准确
4. **允许"各显神通"**：封闭分区确保公平比较，开放分区鼓励创新
5. **保持"与时俱进"**：模块化设计让基准可以跟上 ML 快速发展的步伐

原文翻译

影响。MLPerf Inference（v0.5 版本）于 2019 年 10 月接受测试。我们收到了来自 14 个组织的超过 600 个提交结果，涵盖各种任务、框架和平台。审核测试自动评估了这些提交结果，并确认其中 595 个为有效结果。结果显示了四个数量级的性能变化，范围从嵌入式设备和智能手机到数据中心系统，展示了不同的度量指标和推理场景如何在更稳健地评估 AI 推理加速器方面发挥作用。
保持更新。因为 ML 仍在发展，我们建立了定期维护和更新 MLPerf Inference 的流程。请参见 `mlperf.org` 获取最新的基准测试、规则等。

关键概念

- MLPerf Inference 的五大贡献：代表性工作负载、现实场景、质量目标、宽松规则、模块化方法
- 四种推理场景：单流、多流、服务器、离线
- 两个分区：封闭分区（公平比较）、开放分区（鼓励创新）
- 首次提交：14组织、600+结果、595有效、4个数量级的性能跨度

3 推理基准测试的挑战 (Inference-Benchmarking Challenges)

原文翻译

一个有用的 ML 基准测试必须克服三个关键挑战：模型的多样性、部署场景的多样性以及推理系统的多样性。

3.1 模型的多样性

原文翻译

为基准测试选择正确的模型是困难的；选择取决于应用。例如，自动驾驶汽车中的行人检测比标注动物照片有更高的准确度要求，这是因为错误预测的后果不同。类似地，机器学习推理的服务质量（QoS）要求从几乎没有延迟约束的离线处理到实时应用的毫秒级别，跨越几个数量级。
覆盖这一广阔的设计空间需要仔细选择代表现实场景的模型。但即使对于单个 ML 任务（如图像分类），也有许多模型在准确度和计算复杂度之间呈现不同的权衡，如图 1 所示。这些模型在计算和内存需求方面差异巨大（例如，在 GFLOPS 上有 50 倍的差异），而相应的 Top-1 准确度范围从 55% 到 83%**【引用说明：Bianco2018 对代表性深度神经网络架构的基准分析——提供了图 1 中的数据】**。这种变化创建了帕累托前沿（Pareto frontier）而不是一个最优选择。即使是很小的准确度变化（例如几个百分点）也可能大幅改变计算需求（例如 5--10 倍）。例如，SE-ResNeXt-50**【引用说明：Hu2018 SENet、Xie2017 ResNeXt——两种高级图像分类网络】**和 Xception**【引用说明：Chollet2017 深度可分离卷积网络】**达到大约相同的准确度（~79%），但表现出 2 倍的计算差异。

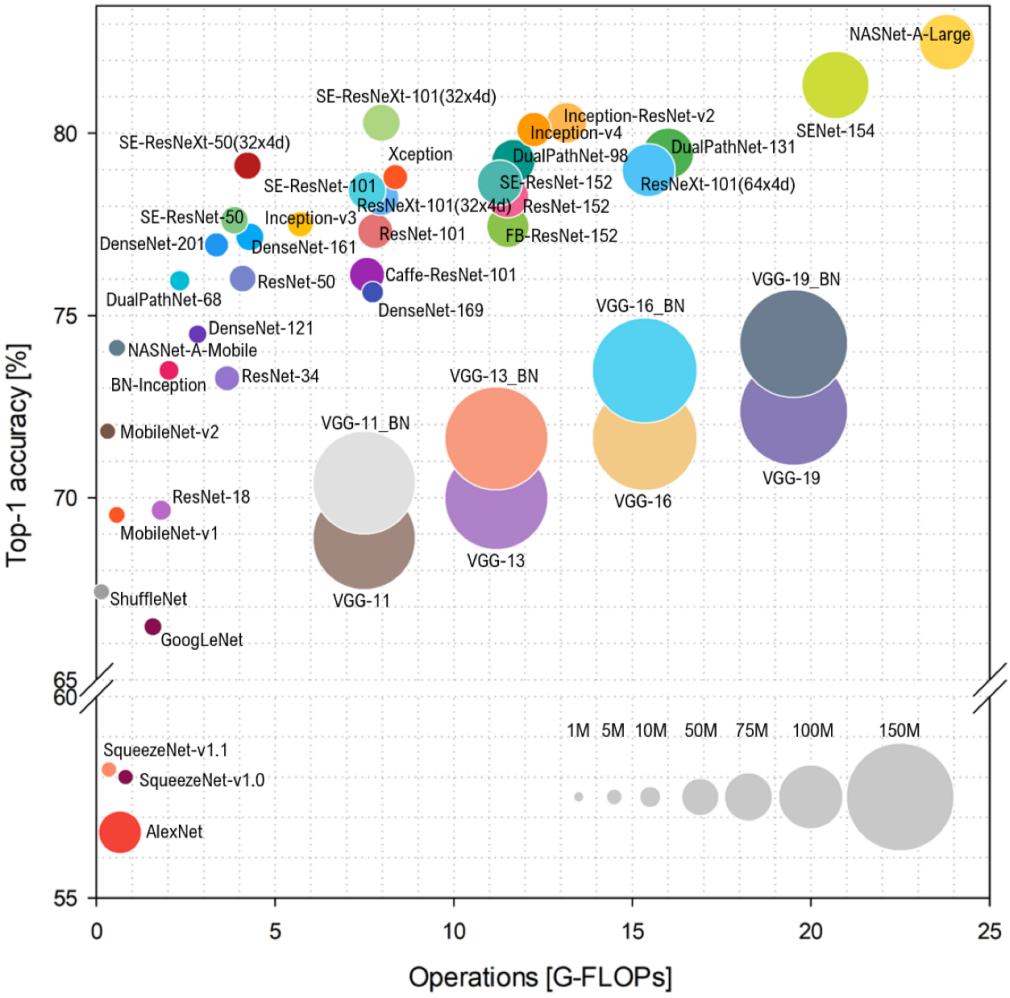

Figure 1: 图像分类 ML 模型多样性的示例（图来自 Bianco2018）。没有单一模型是最优的；每个模型在准确度、内存需求和计算复杂度之间呈现设计权衡。

图 1 详细解读

这张图展示了图像分类领域各种深度学习模型的"性能地图"：
横轴：计算复杂度（GFLOPS，十亿浮点运算数）——越靠右，计算量越大。
纵轴：Top-1 准确度——越高越准确。
气泡大小：模型参数量（内存占用）——越大的气泡需要越多内存。
关键观察：

- 不存在一个"万能"模型——所有模型都在准确度、速度和大小之间权衡
- 形成了"帕累托前沿"：右上方的模型更准确但更慢；左下方的模型更快但不那么准确
- 同样准确的模型可以有很不同的计算成本（例如 SE-ResNeXt-50 和 Xception）
- 这就是为什么基准测试需要同时包含"重量级"和"轻量级"模型

3.2 部署场景的多样性

原文翻译

除了准确度和计算复杂度外，一个具有代表性的 ML 基准测试还必须考虑在各种应用部署场景中输入数据的可用性和到达模式。例如，在离线批处理中，如照片分类，所有数据可能已经在（网络）存储中随时可用，允许加速器达到并维持峰值性能。相比之下，翻译、图像标记和其他任务则根据最终用户流量经历可变的到达模式。
类似地，增强现实和自动驾驶等实时应用处理的是持续的数据流，而不是将所有数据放在内存中。虽然相同的模型架构可以在每个场景中使用，但数据批处理和类似的优化可能不适用，导致性能大不相同。仅计时设备上的推理延迟无法反映真实世界的需求。

通俗解释

这里用一个"餐厅"的类比来理解不同部署场景：

- **离线批处理**就像"提前准备好所有食材做一桌宴席"——你可以从容不迫地安排
- **在线服务**就像"接单即做的外卖"——订单随机到来，你必须快速响应
- **实时处理**就像"自助餐传送带"——数据不断流过来，你必须跟上速度

同一个厨师（同一个模型）在这三种场景下的表现可能完全不同！所以不能只用一种方式来测试。

3.3 推理系统的多样性

原文翻译

推理应用、数据集、模型、机器学习框架、工具集、库、系统和平台的可能组合数量众多，进一步增加了系统化和可重现基准测试的复杂性。图 2 显示了 ML 领域的广度和深度。硬件和软件两方面都表现出相当大的复杂性。

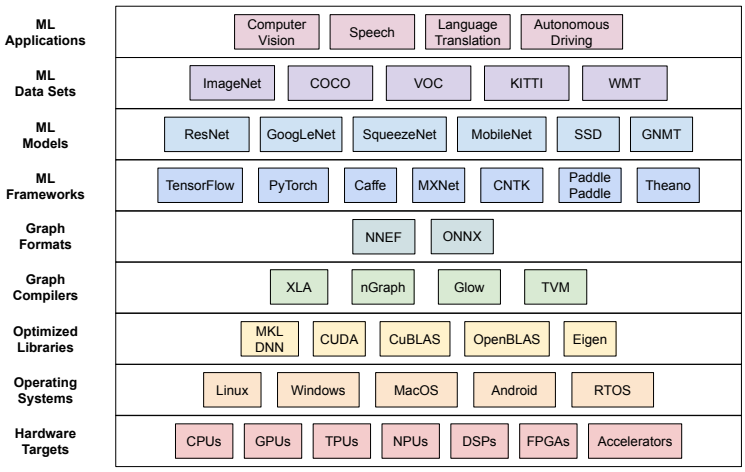

Figure 2: 在技术栈的每个层级上选项的多样性，以及跨层的组合，使得推理系统的基准测试变得困难。

图 2 详细解读

这张图展示了 ML 推理系统的完整技术栈，像一个"千层饼"：
从上到下：

- **应用层**：图像分类、目标检测、翻译等
- **模型层**：ResNet、MobileNet、GNMT 等
- **框架层**：TensorFlow、PyTorch、MXNet 等
- **优化库层**：cuDNN、MKL、TensorRT 等
- **硬件层**：CPU、GPU、DSP、FPGA、ASIC 等

每一层都有多种选择，而且跨层组合还会产生各种兼容性问题。例如文中提到的 NMS 算子在 TensorFlow 和 TensorFlow Lite 中的实现就不同，导致 SSD-MobileNet-v1 的准确度从 23.1% 下降到 22.3% mAP。
这种复杂性是 MLPerf 需要解决的核心挑战之一。

Table 1: MLPerf Inference v0.5 中的 ML 任务。每个任务反映了对大量提交者具有重要意义的商业和研究用例，它们共同覆盖了广泛的计算模式（如 CNN 和 RNN）。

领域	任务	参考模型	数据集	质量目标
视觉	图像分类（重量级）	ResNet-50 v1.5 25.6M 参数 8.2 GOPS/输入	ImageNet (224x224)	FP32精度的99%（76.456%） Top-1
视觉	图像分类（轻量级）	MobileNet-v1 224 4.2M 参数 1.138 GOPS/输入	ImageNet (224x224)	FP32精度的98%（71.676%） Top-1
视觉	目标检测（重量级）	SSD-ResNet-34 36.3M 参数 433 GOPS/输入	COCO (1200x1200)	FP32精度的99%（0.20 mAP）
视觉	目标检测（轻量级）	SSD-MobileNet-v1 6.91M 参数 2.47 GOPS/输入	COCO (300x300)	FP32精度的99%（0.22 mAP）
语言	机器翻译	GNMT ~210M 参数	WMT16 EN-DE	FP32精度的99%（23.9 SacreBLEU）

表 1 解读

这张表列出了 MLPerf v0.5 的五个基准测试任务：

- **图像分类**有两个模型：重量级的 ResNet-50（25.6M参数，8.2 GOPS）适合数据中心，轻量级的 MobileNet-v1（4.2M参数，1.138 GOPS）适合手机——后者参数少 6.1 倍，运算少 6.8 倍
- **目标检测**也有两个模型：重量级 SSD-ResNet-34 用大图（1200x1200），轻量级 SSD-MobileNet 用小图（300x300）
- **机器翻译**用 GNMT（Google NMT），是 RNN 架构，参数量最大（210M）
- **质量目标**基本都是 FP32 参考精度的 99%，即允许不超过 1% 的精度损失

这种"重量级+轻量级"的设计确保了从嵌入式到数据中心的全面覆盖。

原文翻译

在软件方面，大约十几个 ML 框架通常用于开发深度学习模型，如 Caffe/Caffe2**【引用说明：Jia2014 Berkeley开发的深度学习框架】**、Chainer**【引用说明：Tokui2015 日本Preferred Networks开发的框架】**、CNTK**【引用说明：Seide2016 Microsoft的开源深度学习工具包】**、Keras**【引用说明：Chollet2015 高层神经网络API】**、MXNet**【引用说明：Chen2015 分布式ML库】**、TensorFlow**【引用说明：Abadi2016 Google的大规模ML系统】**和 PyTorch**【引用说明：Paszke2017 Facebook的自动微分框架】**。独立地，还有许多优化库，如 cuDNN**【引用说明：Chetlur2014 NVIDIA的深度学习GPU加速原语】**、Intel MKL**【引用说明：Intel数学核心库】**和 FBGEMM**【引用说明：Khudia2018 Facebook的服务器端推理优化GEMM库】**，支持各种推理运行时，如 Apple CoreML**【引用说明：Apple的设备端ML框架】**、Intel OpenVino**【引用说明：Intel的推理优化工具包】**、NVIDIA TensorRT**【引用说明：NVIDIA的可编程推理加速器】**、ONNX Runtime**【引用说明：Bai2019 开放神经网络交换格式运行时】**、Qualcomm SNPE**【引用说明：Qualcomm的神经处理引擎SDK】**和 TF-Lite**【引用说明：Lee2019 面向移动GPU的设备端推理】**。

每种组合都有其特殊性，使得支持最新的神经网络模型架构成为一个挑战。以目标检测的非极大值抑制（NMS）算子为例。在 TensorFlow 中训练目标检测模型时，常规 NMS 算子可以平滑单个对象的不精确边界框。但这个实现在 TensorFlow Lite 中不可用，后者是为移动设备量身定制的，转而实现了快速 NMS。因此，当将模型从 TensorFlow 转换到 TensorFlow Lite 时，SSD-MobileNet-v1 的准确度从 23.1% 下降到 22.3% mAP。这种微妙的差异使得将模型从一个框架精确移植到另一个框架变得困难。

在硬件方面，平台极其多样，从熟悉的处理器（如 CPU、GPU 和 DSP）到 FPGA、ASIC 以及模拟和混合信号处理器等新型加速器。每个平台都有特定于硬件的特性和约束，根据模型和场景可能启用或中断性能。

注意事项

NMS（非极大值抑制）算子的跨框架移植问题是一个典型案例：即使是同一个模型，在不同框架/运行时中的行为可能不完全一致。SSD-MobileNet-v1 从 TF 到 TF-Lite 的移植导致了 0.8% mAP 的精度损失（23.1% → 22.3%）。这说明"同一个模型"在不同环境下可能并不等价——这正是 MLPerf 需要明确定义"等价实现"规则的原因。

4 基准测试设计 (Benchmark Design)

原文翻译

将模型多样性与软件系统的范围相结合，为推导满足行业需求的稳健 ML 基准测试带来了独特的挑战。为了克服这一挑战，我们基于社区输入采用了一套原则来开发灵活而稳健的产品。在本节中，我们描述基准测试、质量目标以及可以评估 ML 系统的场景。

4.1 代表性、广泛可访问的工作负载

原文翻译

设计 ML 基准测试与设计传统非 ML 基准测试不同。MLPerf 定义了机器学习系统可以执行的高层任务（如图像分类）。对于每个任务，我们在几个广泛使用的框架中提供一个或多个规范参考模型。任何与参考模型数学等价的实现都被视为有效的，某些其他偏差（如数值格式）也是允许的。例如，全连接层可以用不同的缓存分块和评估策略来实现。因此，提交的结果需要优化才能获得良好的性能。

参考模型和一类有效的等价实现给大多数 ML 系统提供了自由，同时仍然允许相关比较。MLPerf 使用 32 位浮点权重提供参考模型，并且为了方便，提供了精心实现的等价模型以适配三种格式：TensorFlow、PyTorch 和 ONNX**【引用说明：分别对应三种主流ML框架格式】**。

如表 1 所示，我们选择了一组初始的视觉和语言任务及相关参考模型。视觉和翻译在从边缘设备到云数据中心的计算系统中广泛使用。成熟且表现良好的、具有不同架构（如 CNN 和 RNN）的参考模型也已可用。

原文翻译

图像分类。许多商业应用使用图像分类，它是评估 ML 系统性能的事实标准。分类器网络接收一张图像并选择最能描述它的类别。示例应用包括照片搜索、文本提取和工业自动化（如物体分类和缺陷检测）。我们使用 ImageNet 2012 数据集**【引用说明：Deng2009 ImageNet——大规模层次图像数据库，计算机视觉领域的标准基准数据集】**，在预处理中将图像裁剪为 224x224，并测量 Top-1 准确度。

我们选择了两个模型：一个计算量较大但更准确的重量级模型和一个更快但不太准确的轻量级模型。重量级模型 ResNet-50 v1.5**【引用说明：He2016 深度残差网络——图像识别领域最常用的基准网络】**直接来自 MLPerf Training 套件以保持一致性。ResNet-50 是最常用于性能声称的网络。不幸的是，它有多个微妙不同的实现，使得大多数比较变得困难。我们特别选择了 ResNet-50 v1.5 以确保有用的比较和跨主要框架的兼容性。这个网络表现出良好的可重现性，使其成为低风险的选择。

轻量级模型 MobileNet-v1-224**【引用说明：Howard2017 MobileNets——高效的移动端卷积网络】**使用更小的深度可分离卷积来降低复杂性和计算负担。MobileNet 网络提供不同的计算和准确度选项——我们选择了全宽、全分辨率的 MobileNet-v1-1.0-224。与 ResNet-50 v1.5 相比，它将参数减少了 6.1 倍，运算减少了 6.8 倍。我们为 MLPerf Inference v0.5 套件评估了 MobileNet-v1 和 MobileNet-v2**【引用说明：Sandler2018 MobileNetV2——倒残差和线性瓶颈】**，由于前者更广泛的采用而选择了它。

原文翻译

目标检测。目标检测是一种视觉任务，确定图像中物体周围边界框的坐标，然后对这些框进行分类。实现通常使用预训练的图像分类器网络作为骨干或特征提取器，然后执行回归进行定位和边界框选择。目标检测对于汽车任务（如检测危险和分析交通）和移动零售任务（如识别图片中的物品）至关重要。我们选择了 COCO 数据集**【引用说明：Lin2014 Microsoft COCO——常见物体上下文数据集】**，包含轻量级和重量级两个模型。

类似于图像分类，我们选择了两个模型。我们的小模型使用 300x300 图像尺寸，这是智能手机和其他紧凑设备的典型分辨率。对于较大的模型，我们将数据集放大以更接近高清图像传感器的输出（总共 1.44 MP）。选择较大输入尺寸是基于社区反馈，特别是来自汽车和工业自动化客户。目标检测的质量度量是平均精确率均值（mAP）。

重量级目标检测器的参考模型是 SSD**【引用说明：Liu2016 Single Shot MultiBox Detector——单次检测方法】**搭配 ResNet-34 骨干，它也来自我们的训练基准测试。轻

量级目标检测器的参考模型使用 MobileNet-v1-1.0 骨干，这在受限计算环境中更典型。我们根据移动和嵌入式社区的反馈选择了 MobileNet 特征检测器。

原文翻译

翻译。神经机器翻译（NMT）在自然语言处理中很流行。NMT 模型将源语言的词序列翻译为目标语言，出现在翻译应用和服务中。我们的翻译数据集是 WMT16 EN-DE**【引用说明：WMT2016 机器翻译大会——标准的翻译基准数据集】**。质量度量是 BLEU（Bilingual Evaluation Understudy）分数**【引用说明：Papineni2002 BLEU——自动机器翻译评估方法，NMT领域最常用的度量】**，使用 SacreBLEU 实现**【引用说明：Post2018 呼吁BLEU分数报告的清晰度——推荐使用SacreBLEU以确保可重现】**。我们的参考模型是 GNMT**【引用说明：Wu2016 Google的神经机器翻译系统——弥合人类翻译与机器翻译的差距】**，它采用了一种成熟的循环神经网络（RNN）架构，并且是训练基准测试的一部分。RNN 对于序列和时间序列数据很流行，因此包含 GNMT 确保我们的参考套件捕获了各种计算模式。

4.2 稳健的质量目标

原文翻译

质量和性能对于所有形式的机器学习都是密切相关的。虽然推理的起点是达到目标质量的预训练参考模型，但许多系统架构可以牺牲模型质量来降低延迟、降低总拥有成本（TCO）或提高吞吐量。准确度、延迟和 TCO 之间的权衡是特定于应用的。用 1% 的模型准确度换取 50% 更低的 TCO，在识别猫咪照片时是明智的，但在在线行人检测中则不那么明智。为了反映这一重要方面，我们建立了每个模型的质量目标。

我们要求几乎所有实现都达到 FP32 参考模型准确度 1% 以内的质量目标。（例如，ResNet-50 v1.5 模型达到 76.46% Top-1 准确度，等价模型必须至少达到 75.70% Top-1 准确度。）然而，初始实验表明，对于以移动为重点的网络——MobileNet 和 SSD-MobileNet——在不重训练的情况下，准确度损失是不可接受的。由于准确度低，性能基准测试将变得不具代表性，我们无法继续。

为了解决准确度下降问题，我们采取了三个步骤。首先，我们为量化友好的权重训练了 MobileNet 模型，使我们能够将质量窗口缩小到 2%。其次，为了降低基于 MobileNet 的提交的训练敏感性，我们提供了量化为 8 位整数格式的等价 MobileNet 和 SSD-MobileNet 实现。第三，对于 SSD-MobileNet，我们将质量要求降低到 22.0 mAP 以应对使用 MobileNet 骨干的挑战。

为了提高提交的可比较性，我们禁止重训练。我们之前的经验和可行性研究确认，对于 8 位整数算术——这是许多系统的预期部署路径——约 1% 的相对准确度目标在不重训练的情况下很容易实现。

通俗解释

质量目标的设计体现了工程上的务实精神：

1. **基本规则**：允许最多 1% 的精度损失（相对于 FP32 参考）。这主要为了让 INT8 量化的系统可以参与。
2. **MobileNet 例外**：轻量级模型在量化后精度损失太大，所以采取了特殊措施——提供量化友好的权重、预量化的 INT8 模型、放宽质量要求到 2%。
3. **禁止重训练**：如果允许重训练，不同提交者的训练能力差异会影响基准测试的公平性。

这里的设计智慧在于：既要设定足够高的质量门槛以确保实用性，又不能把门槛设得太高以至于排除合法的优化技术（如 INT8 量化）。

4.3 现实的端用户场景

Table 2: 场景描述与度量指标。每个场景基于客户和供应商的输入，针对一个现实世界用例。

场景	查询生成	度量指标	样本数/查询	示例
单流 (SS)	顺序	第90百分位延迟	1	输入自动补全，实时AR
多流 (MS)	带丢弃的到达间隔	受延迟约束的流数量	N	多摄像头驾驶辅助，大规模自动化
服务器 (S)	泊松分布	受延迟约束的每秒查询数	1	翻译网站
离线 (O)	批处理	吞吐量	至少 24,576	照片分类

原文翻译

ML 应用有各种使用模型和许多优值指标，这反过来需要多个性能度量指标。例如，对视频摄像头输出进行分类的图像识别系统的优值指标与基于云的翻译系统完全不同。
为了应对这些场景，我们调查了 MLPerf 的广泛成员，包括客户和供应商。基于该反馈，我们确定了四种代表许多关键推理应用的场景：**单流**、**多流**、**服务器**和**离线**。这些场景模拟了移动设备、自动驾驶汽车、机器人和基于云设置的 ML 工作负载行为（表 2）。

原文翻译

单流（Single-stream）。单流场景代表具有查询样本大小为 1 的单个推理查询流，反映了许多响应性至关重要的客户端应用。一个例子是在 Google Pixel 4 智能手机上的离线语音转录。为了测量性能，我们向推理系统注入一个查询；当查询完成时，我们记录完成时间并注入下一个查询。度量指标是查询流的第 90 百分位延迟。

原文翻译

多流（Multistream）。多流场景代表具有查询流的应用，但每个查询包含多个推理，反映了各种工业自动化和遥感任务。例如，许多自动驾驶汽车同时分析来自多个摄像头的帧。为了模拟并发场景，我们以固定时间间隔（如 50 ms）发送包含 N 个输入样本的新查询。该间隔是特定于基准测试的，也充当延迟边界，范围从 50 到 100 毫秒。如果系统可用，它处理传入的查询。如果它仍在一个间隔中处理先前的查询，它跳过该间隔并将剩余查询延迟一个间隔。不超过 1% 的查询可以产生一个或多个跳过的间隔。查询的 N 个输入样本在内存中是连续的，这准确反映了生产输入管道，避免惩罚那些否则需要将样本复制到连续内存区域才能开始推理的系统。性能度量是系统在满足 QoS 要求的同时支持的流的整数数量。

原文翻译

服务器（Server）。服务器场景代表查询到达是随机的且延迟很重要的在线应用。几乎每个面向消费者的网站都是一个好例子，包括百度、Google 和 Microsoft 的在线翻译等服务。对于此场景，查询各有一个样本，遵循泊松分布。被测系统在特定于基准测试的延迟边界内响应每个查询，该边界从 15 到 250 毫秒不等。对于视觉任务，不超过 1% 的查询可以超过延迟边界；对于翻译，不超过 3%。服务器场景的性能度量是在满足 QoS 要求的同时可实现的泊松参数，即每秒查询数（QPS）。

原文翻译

离线（Offline）。离线场景代表所有数据立即可用且延迟不受约束的批处理应用。一个例子是识别相册中的人物和地点。对于此场景，我们发送一个包含所有要处理的样本数据 ID 的单个查询，系统可以按任何顺序处理输入数据。与多流场景类似，查询中相邻的样本在内存中是连续的。离线场景的度量是以每秒样本数衡量的吞吐量。

通俗解释

四种场景用生活中的类比：

1. **单流→"一问一答"**：就像你在手机上用语音助手——说一句话，等回答，再说下一句。度量：响应有多快（第90百分位延迟）
2. **多流→"监控室"**：就像一个保安同时盯着 N 个监控屏幕——每 50ms 所有摄像头同时来一帧，你得全部及时处理。度量：你最多能同时盯几个屏幕
3. **服务器→"网上翻译"**：用户随机发来翻译请求（泊松分布），你得在延迟限制内回复。度量：你每秒能处理多少请求（QPS）
4. **离线→"批改试卷"**：所有试卷一次性堆在桌上，你想怎么改就怎么改，不限时间。度量：你每秒能改多少份

Table 3: 多流和服务器场景中的延迟约束。

任务	多流到达时间	服务器QoS约束
图像分类（重量级）	50 ms	15 ms
图像分类（轻量级）	50 ms	10 ms
目标检测（重量级）	66 ms	100 ms
目标检测（轻量级）	50 ms	10 ms
机器翻译	100 ms	250 ms

表 3 解读

延迟约束反映了不同任务的实际部署需求：

- **多流到达时间**：对应帧率要求。50ms = 20 Hz，66ms ≈ 15 Hz。这是自动驾驶等实时

应用的最低帧率要求。

- **服务器QoS**：图像分类要求 10--15ms（因为通常是用户等待链中的一个环节），目标检测允许 100ms（单次处理更复杂），翻译允许 250ms（用户对翻译延迟的容忍度更高）。
- 轻量级模型的延迟约束（10ms）比重量级（15ms）更严格，因为轻量级模型主要部署在对延迟要求更高的移动端。

原文翻译

对于多流和服务器场景，延迟是系统行为的关键组成部分，约束着各种性能优化。例如，大多数推理系统需要最小（且特定于架构的）批大小来充分利用底层计算资源。但服务器中的查询到达率是随机的，因此它们必须优化尾延迟，并可能以次优批大小处理推理。
表 3 显示了 MLPerf Inference v0.5 中每个任务的延迟约束。与基准测试的其他方面一样，我们基于可行性和社区协商选择了这些约束。多流场景中大多数视觉任务的到达时间对应 15--20 Hz 的帧率，这是许多应用的最低要求。服务器场景的 QoS 约束来自给定总体用户延迟目标的推理定时预算估计。

4.4 统计置信的尾延迟边界

原文翻译

每个任务和场景组合需要最少数量的查询，以确保结果在统计上是稳健的，并充分捕获稳态系统行为。该数量由尾延迟百分位、期望的余量和期望的置信区间决定。置信度是延迟边界在报告结果的特定余量内的概率。我们选择了 99% 的置信边界，并将余量设置为远小于尾延迟百分比和 100% 之差的值。从概念上讲，该余量应该相对较小。因此，我们的选择是尾延迟百分比和 100% 之差的二十分之一。公式如下：

数学推导详解

公式推导：最小查询数量的计算
公式 1：余量（Margin）的计算

$$Margin = \frac{1 - TailLatency}{20} \tag{1}$$

符号含义：

- $TailLatency$：尾延迟百分位（如 0.90 表示第90百分位）
- $Margin$：可接受的误差余量
- 除以 20 是设计选择，使余量"远小于"$1 - TailLatency$

数值例子：若 $TailLatency = 0.90$（第90百分位），则 $Margin = \frac{1-0.90}{20} = \frac{0.10}{20} = 0.005$（即 0.5%）。
公式 2：所需查询数量

$$NumQueries = \left(Normsinv\left(\frac{1 - Confidence}{2}\right)\right)^2 \times \frac{TailLatency \times (1 - TailLatency)}{Margin^2} \tag{2}$$

符号含义：

- $Confidence$：置信水平（这里是 0.99，即 99%）
- $Normsinv(\cdot)$：标准正态分布的逆函数（即 z 值）
- $NumQueries$：需要的最少查询数量

完整数值例子（第90百分位、99%置信度）：

$$
\begin{aligned}
Margin &= \frac{1-0.90}{20} = 0.005 \\
z &= Normsinv\left(\frac{1-0.99}{2}\right) = Normsinv(0.005) = -2.576 \\
NumQueries &= (-2.576)^2 \times \frac{0.90 \times 0.10}{0.005^2} \\
&= 6.636 \times \frac{0.09}{0.000025} \\
&= 6.636 \times 3600 = 23{,}886
\end{aligned}
$$

然后向上取整到最近的 $2^{13} = 8192$ 的倍数：$3 \times 2^{13} = 24{,}576$。
统计学原理：这与选举民调中确定合适样本量的数学完全相同**【引用说明：Tamhane2000 统计学与数据分析——提供了确定样本量所需的统计学理论基础】**。给定无限选民（对应无限多可能的查询），三个变量决定样本量：尾延迟百分比（对应民调比例）、置信度和余量。

通俗解释

简单来说，这个统计计算回答了一个问题：**需要运行多少次测试，才能确信结果是可靠的?**
就像选举前的民调：你想知道某候选人的支持率是 90%，要有 99% 的把握这个数字误差不超过 0.5%，那你至少需要调查约 24,000 人。
同理：你想确认系统的第90百分位延迟满足要求，有 99% 的把握误差在 0.5% 以内，你至少需要运行约 24,000 次查询。
而对于更严格的第99百分位要求（如服务器场景），需要的查询数量急剧增加到约 270,000 次——这就是"高度非线性"的体现。

Table 4: 统计置信度的查询需求。所有结果必须满足最低 LoadGen 场景要求。

尾延迟百分位	置信区间	误差余量	推理次数	取整后推理次数
90%	99%	0.50%	23,886	$3 \times 2^{13} = 24{,}576$
95%	99%	0.25%	50,425	$7 \times 2^{13} = 57{,}344$
99%	99%	0.05%	262,742	$33 \times 2^{13} = 270{,}336$

Table 5: 每个任务的查询数和每查询样本数。

模型	查询数 / 每查询样本数			
	单流	多流	服务器	离线
图像分类（重量级）	1K / 1	270K / N	270K / 1	1 / 24K
图像分类（轻量级）	1K / 1	270K / N	270K / 1	1 / 24K
目标检测（重量级）	1K / 1	270K / N	270K / 1	1 / 24K
目标检测（轻量级）	1K / 1	270K / N	270K / 1	1 / 24K
机器翻译	1K / 1	90K / N	90K / 1	1 / 24K

表 4 和表 5 解读

两张表的关系：

- 表 4 说明了"为什么需要这么多查询"——从统计学角度推导
- 表 5 说明了"每个任务实际需要多少查询"
- 单流场景只需 1K 查询（因为只看第90百分位，且要让嵌入式设备和手机能快速完成）
- 多流和服务器场景对视觉任务需要 270K 查询（第99百分位要求），对翻译需要 90K（第97百分位）
- 离线场景只需 1 个查询，但包含至少 24K 个样本
- 多流场景的总样本数是 $N \times 270K$（N 是流数量），所以运行时间很长（2.5--7小时）

原文翻译

99百分位保证需要 262,742 个查询，向上取整到 33×2^{13}，即 270K。对于多流和服务器场景，对视觉任务需要 270K 查询，如表 5 所示。因为多流基准测试将处理每个查询 N 个样本，总样本数将是 $N\times$ 270K。机器翻译有 97 百分位延迟保证，只需要 90K 查询。为了可重复性，我们多次运行多流和服务器场景。但多流场景的到达率和查询数保证了 2.5 到 7.0 小时的运行时间。为了在可重复性和运行时间之间取得平衡，我们要求服务器场景运行五次，结果取这五次中的最小值。其他场景只需一次运行。我们预计在未来的 MLPerf 版本中重新审视这一选择。
所有基准测试还必须至少运行 60 秒，并根据场景需要处理额外的查询和/或样本。最小运行时间确保我们测量电源管理系统和支持动态电压频率调节（DVFS）系统的均衡行为，特别是对于查询较少的单流场景。

5 推理提交系统 (Inference Submission System)

原文翻译

一个 MLPerf Inference 提交系统包含一个被测系统（SUT）、负载生成器（LoadGen）、一个数据集和一个准确度脚本。在本节中我们描述这些各种组件。图 3 显示了推理系统的概述。数据集、LoadGen 和准确度脚本对所有提交都是固定的，由 MLPerf 提供。提交者有很大的自由度可以根据其架构的需求和工程判断来实现 SUT。通过在提交者所有和 MLPerf 所有的组件之间建立明确的边界，基准测试在提交之间保持了可比较性。

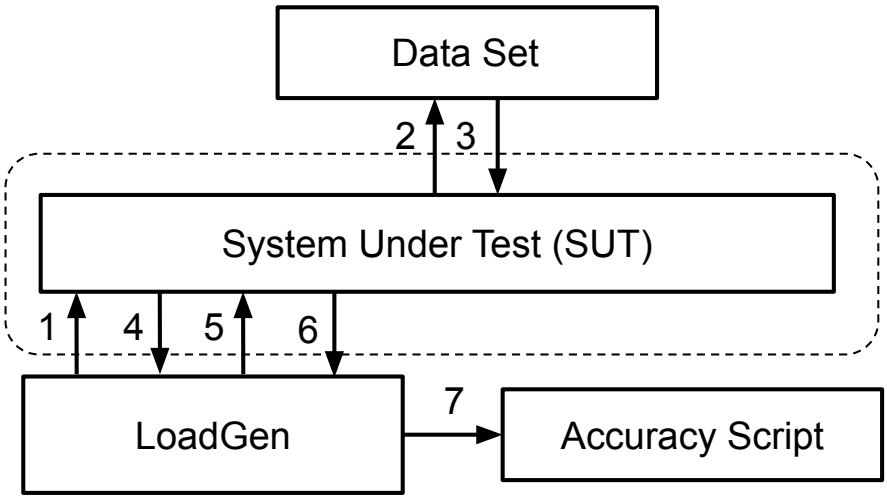

Figure 3: MLPerf Inference 被测系统（SUT）及相关组件。首先，LoadGen 请求 SUT 加载样本 (1)。SUT 然后将样本加载到内存中 (2--3) 并在准备就绪时向 LoadGen 发出信号 (4)。接下来，LoadGen 向 SUT 发出请求 (5)。基准测试处理结果并将其返回给 LoadGen (6)，然后 LoadGen 输出日志供准确度脚本读取和验证 (7)。

图 3 详细解读

这张图展示了 MLPerf 推理测试的完整工作流程，像一个"考试流程"：

步骤 1-4（准备阶段，不计时）：

- (1) LoadGen（出题者）告诉 SUT（考生）："准备好这些题目"
- (2-3) SUT 将数据加载到内存（考生把资料摆好）
- (4) SUT 告诉 LoadGen："我准备好了"

步骤 5-7（考试阶段，计时）：

- (5) LoadGen 按场景规则发出查询（发考题）
- (6) SUT 处理并返回结果（答题）
- (7) LoadGen 记录日志，准确度脚本验证结果（阅卷）

关键设计：LoadGen 是 MLPerf 统一提供的，确保所有"考生"用同一套"出题和计时系统"，这是公平性的基础。

5.1 被测系统

原文翻译

提交者负责被测系统。MLPerf Inference 的目标是测量各种架构的系统性能。但现实性、可比较性、架构中立性和对小型提交团队的友好性需要仔细权衡。例如，一些部署拥有编译器、计算机架构和机器学习专家团队，他们积极地协同优化训练和推理系统，以在大规模全球客户基础上实现成本、准确度和延迟目标。一个不受约束的基准测试会使经验较少、ML 训练资源较少的公司处于不利地位。
因此，我们设置了模型等价规则，允许提交者为了效率在不同架构上重新实现模型。规则提供了禁止技术的完整列表和允许技术的示例列表。我们选择了显式黑名单来鼓励广泛的技术范围并支持架构多样性。示例列表说明了黑名单的边界，同时也鼓励常见和适当的优化。

原文翻译

允许的技术。允许技术的示例包括：任意数据排列以及权重的不同输入和内存表示；数学等价变换；近似（如用多项式替换超越函数）；在场景限制内的乱序查询处理；用数学等价的稀疏操作替换密集操作；融合和拆分操作；以及在一个或多个批大小之间动态切换。
为了促进架构和应用中立性，我们采用了不计时预处理。实现可以将其输入转换为特定于系统的理想形式作为不计时操作。理想情况下，全系统基准测试应捕获所有与性能相关的操作。然而在 MLPerf 中，我们明确允许不计时预处理。不存在供应商中立或应用中立的预处理。例如，具有集成摄像头的系统可以使用硬件/软件协同设计来确保图像以理想格式到达内存；而从互联网接受 JPEG 的系统则不能。
我们还允许并支持量化到许多不同的数值格式以确保架构中立性。提交者提前注册其数值格式以帮助指导准确度目标讨论。批准的列表包括 INT4、INT8、INT16、UINT8、UINT16、FP11（1位符号、5位尾数和5位指数）、FP16、bfloat16 和 FP32。量化到低精度格式通常需要校准以确保足够的推理质量。对于每个参考模型，MLPerf 提供了一个可用于校准量化网络的小型固定数据集。此外，它还提供了预量化为 INT8 的 MobileNet 版本，因为不经重训练（我们禁止重训练）准确度会大幅下降。

原文翻译

禁止的技术。我们禁止重训练和剪枝以确保可比较性。虽然这一限制在某些情况下可能无法反映实际部署，但使用参考权重（可能经过校准）和最低准确度目标的连锁要求对可比较性最为重要。我们可能最终会放宽这一限制。
为了简化基准测试评估，我们禁止缓存。在实践中，推理系统会缓存查询。例如，``I love you'' 是 Google Translate 最频繁的查询之一，但该服务并不每次都从头翻译该短语。然而，在基准测试中现实地建模缓存是困难的，因为缓存命中率随应用而显著变化。此外，我们的数据集相对较小，大型系统可以轻松地将它们全部缓存在内存中。
我们还禁止特定于基准测试或数据集的、不适用于生产环境的优化。例如，真实的查询流量是不可预测的，但对于基准测试，流量模式由伪随机数生成器种子预先确定。利用固定查询数量或考虑 LoadGen 实现的优化是被禁止的。类似地，任何利用性能或准确度数据集统计信息的优化也是被禁止的。

关键概念

MLPerf 的"游戏规则"总结:
允许（白名单示例）：数据重排、数学等价变换、近似计算、操作融合/拆分、动态批大小、量化（INT4/8/16, FP16, bfloat16等）、不计时预处理
禁止（黑名单）：重训练、剪枝、缓存、利用基准测试特定信息的优化
设计哲学：使用显式黑名单而非白名单——"除了明确禁止的，其他都允许"。这鼓励创新，同时维护公平性。

5.2 负载生成器（LoadGen）

原文翻译

LoadGen 是 MLPerf Inference 的流量生成器，它加载 SUT 并测量性能。其行为由运行开始时读取的配置文件控制。LoadGen 根据先前描述的场景规则（即单流、多流、服务器和离线）产生查询流量。此外，它收集信息用于日志记录、调试和后处理数据。它记录来自 SUT 的查询和响应，在运行结束时报告统计数据、总结结果并确定运行是否有效。图 4 显示了 LoadGen 如何为每个场景创建查询流量。在服务器场景中，它按照泊松分布发出查询以模拟服务器的查询到达率。在单流情况下，它向 SUT 发出一个查询，等待该查询完成后再发出另一个。

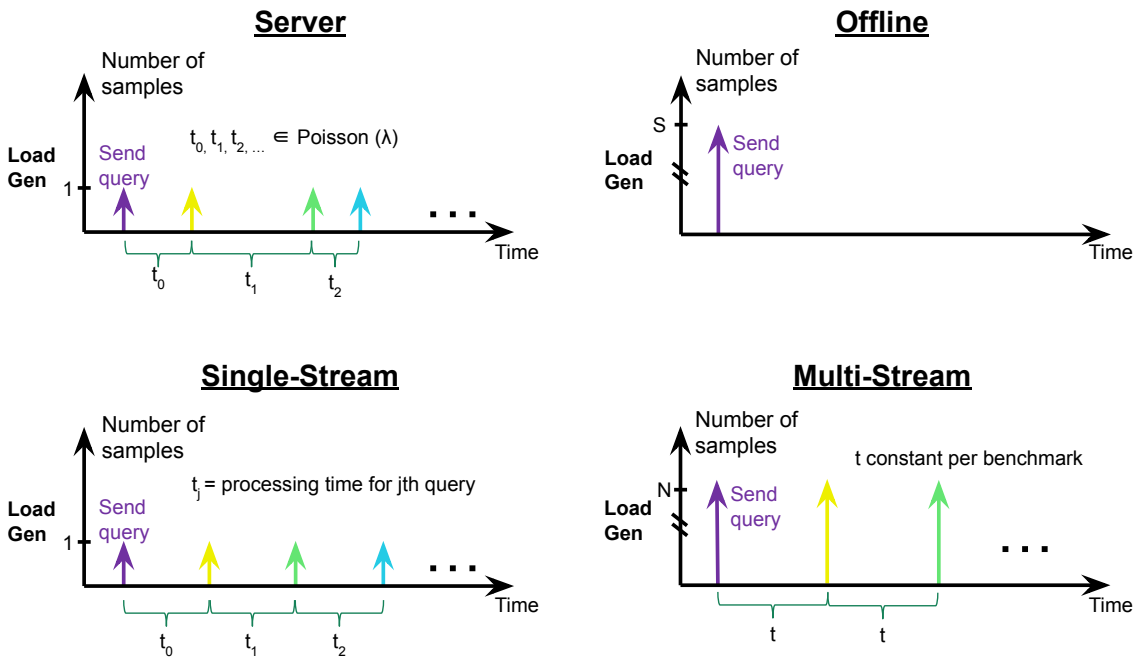

Figure 4: 来自 LoadGen 的查询的时序和数量。

图 4 详细解读

这张图展示了四种场景下 LoadGen 发出查询的模式：

- **单流**：发一个→等完成→再发一个。像打乒乓球：你打一个，对方接住回来，你再打。
- **多流**：每隔固定时间（如50ms）发出包含N个样本的查询。像工厂流水线上每隔固定

时间来一批产品。

- **服务器**：查询按泊松分布随机到达。像餐厅的顾客——你不知道下一个顾客什么时候来。
- **离线**：一次性把所有数据发出。像期末考试——所有题目一起发下来。

每种模式对应不同的性能特征，系统需要不同的优化策略才能表现良好。

原文翻译

在启动时，LoadGen 请求 SUT 将数据集样本加载到内存中。SUT 可以将它们加载到 DRAM 中作为不计时操作，并按规则规定执行其他计时操作。这些不计时操作包括但不限于编译、缓存预热和预处理。SUT 在准备好接收第一个查询时向 LoadGen 发出信号；查询是对一个或多个样本的推理请求。LoadGen 根据选定的场景向 SUT 发送查询。根据场景，它可以一次提交一个查询、以固定间隔提交或按泊松分布提交。SUT 对每个查询运行推理并将响应发送回 LoadGen，后者要么记录响应要么丢弃它。运行结束后，准确度脚本检查记录的响应以确定模型准确度是否在容差范围内。

LoadGen 有两种主要操作模式：**准确度**（accuracy）模式和**性能**（performance）模式。两者都是验证 MLPerf 提交所必需的。在准确度模式下，LoadGen 遍历 ML 任务的整个数据集。模型的任务是在完整数据集上运行推理。之后，准确度结果出现在日志文件中，确保模型满足所需的质量目标。在性能模式下，LoadGen 避免遍历整个数据集，因为系统的性能可以通过对其施加足够的数据集样本来确定。

我们设计 LoadGen 以灵活处理基准测试套件的变化。MLPerf Inference 在 SUT 和 LoadGen 之间有一个接口，以便它可以处理 LoadGen 中的新场景和实验，并将它们推出到所有模型和 SUT，无需额外工作。这样做还有助于合规性和审计，因为关于查询到达、时序和准确度的许多技术规则在提交者代码之外实现。我们通过将 LoadGen 与基准测试及内部表示（如模型、场景、质量和延迟度量）解耦来实现这一目标。LoadGen 实现是一个独立的 C++ 模块。

解耦允许 LoadGen 支持各种语言绑定，允许任何语言的基准测试实现。LoadGen 支持 Python、C 和 C++ 绑定；可以添加额外的绑定。将 LoadGen 与基准测试解耦的另一个好处是 LoadGen 可以扩展以支持更多场景，如多租户模式，其中 SUT 必须持续服务多个模型同时维护 QoS 约束。

此外，将性能测量代码放在提交者代码之外符合 MLPerf 端到端系统基准测试的目标。因此 LoadGen 测量的是整个 SUT 的整体性能，而不是任何单个部分。最后，这一条件增强了基准测试的现实性：推理引擎通常作为更大系统的黑盒组件。

5.3 数据集

原文翻译

我们使用标准且公开可用的数据集以确保社区可以参与。我们不直接托管它们。相反，MLPerf 在 LoadGen 使用数据集运行基准测试之前下载数据集。表 1 列出了我们为每个基准测试选择的数据集。

5.4 准确度检查器

原文翻译

LoadGen 还具有确保提交系统符合规则的功能。此外，它可以自检以确定其源代码在提交过程中是否被修改。为了便于验证，提交者提供一个实验配置文件，允许使用非默认的 LoadGen 功能。详情见第 6.2 节。

6 提交系统评估 (Submission-System Evaluation)

原文翻译

在本节中，我们描述提交、审查和报告流程。参与者可以向不同的分区和类别提交结果。所有提交都经过同行审查以验证有效性。最后，我们描述结果如何报告。

6.1 结果提交、分区和类别

原文翻译

结果提交包含有关 SUT 的信息：性能分数、基准测试代码、突出 SUT 主要配置特征（如加速器数量、CPU 数量、软件版本和内存系统）的系统描述文件，以及详细说明一组任务和场景组合的性能和准确度运行的 LoadGen 日志文件。所有这些数据上传到公共 GitHub 仓库，以便在发布前进行同行审查和验证。
MLPerf Inference 是一个确保广泛覆盖的任务和场景套件，但提交可以包含其子集。许多传统基准测试（如 SPEC CPU）要求提交所有组件。这种方法对运行任意代码的通用处理器是合理的，但 ML 系统通常是高度专用的。
分区。MLPerf Inference 有两个提交结果的分区：封闭和开放。参与者可以向其中一个或两个提交，但必须使用相同的数据集。
封闭分区允许不同系统之间的比较。提交者使用相同的模型、数据集和质量目标，以确保在架构差异巨大的情况下的可比较性。该分区要求预处理、后处理和与参考实现等价的模型。它还允许量化校准（使用我们提供的校准数据集），并禁止重训练。
开放分区促进 ML 系统、算法、优化和硬件/软件协同设计的创新。参与者仍然必须执行相同的 ML 任务，但可以更改模型架构和质量目标。该分区允许任意的前后处理和任意模型，包括重训练等技术。通常，提交结果既不能直接与彼此比较，也不能与封闭分区的提交比较。每个开放分区提交必须包含关于其如何偏离封闭分区的文档。
类别。参照 MLPerf Training，参与者根据硬件和软件可用性将其提交分类为三个类别之一：可用（available）；预览（preview）；以及研究、开发或其他系统（RDO）。这种分类帮助消费者确定系统的成熟度以及它们是否随时可以获取（租用或购买）。

通俗解释

MLPerf 的分区和类别设计非常巧妙：
分区（Division）——"考试的两种模式"：

- **封闭分区**→"标准化考试"：大家用同样的模型、数据、质量要求，只能在实现上优化。适合公平比较。
- **开放分区**→"创新大赛"：可以用不同模型、不同技术，展示各种可能性。适合探索前沿。

类别（Category）——"产品成熟度标签"：

- **可用**：已经上市可以买到的产品
- **预览**：180天内将上市的产品
- **RDO**：研究性质的原型系统

6.2 结果审查

原文翻译

推理系统基准测试的一个挑战是许多系统包含专有和闭源组件（如推理引擎和量化流程），这使得同行审查变得困难。为了在确保可重现结果且没有常见错误的同时适应这些系统，我们开发了一个验证套件来协助同行审查。这些验证工具执行实验，帮助确定提交是否符合定义的规则。

准确度验证。此测试的目的是确保性能模式下的有效推理。默认情况下，推理系统返回给 LoadGen 的结果不会被记录，因此不会被检查准确度。这一选择减少或消除了处理开销，以允许对推理系统性能的准确测量。在此测试中，从 SUT 返回到 LoadGen 的结果被随机记录。日志与准确度模式生成的日志进行比对以确保一致性。

即时缓存检测。LoadGen 通过从数据集中有放回地随机选择查询样本来产生查询，推理系统可能收到具有重复样本的查询。对于相对于数据集大小处理许多样本的高性能系统来说，这种重复是很可能的。为了代表现实部署，规则禁止缓存查询和中间数据。测试有两部分：第一部分生成具有唯一样本索引的查询，第二部分生成具有重复样本索引的查询。它测量每种情况下的性能。检测缓存的方法是确定具有重复样本索引的测试是否比具有唯一样本索引的测试运行显著更快。

替代随机种子测试。通常，LoadGen 基于固定随机种子产生查询。基于该种子的优化是被禁止的。替代随机种子测试用替代种子替换官方随机种子并测量结果性能。

自定义数据集。除了 LoadGen 的验证功能外，我们使用自定义数据集来检测结果缓存。MLPerf Inference 通过用自定义数据集替换参考数据集来验证此行为。我们测量系统在后者上运行的质量和性能，并将结果与在前者上运行的结果进行比较。

6.3 结果报告

原文翻译

MLPerf Inference 不提供"总分"。基准测试工作通常会引发将复杂系统的能力提炼为单个数字从而实现不同系统比较的强烈愿望。但并非所有 ML 任务对所有系统都同样重要，而更重视某些任务的工作高度主观。在最好的情况下，加权和汇总由提交者根据客户需求驱动，因为某些系统可能是为特定 ML 任务设计的。例如，一个系统可能针对视觉而非翻译进行了高度优化。在这种情况下，对所有任务的结果取平均值毫无意义，因为提交者可能并未针对那些市场。

注意事项

MLPerf 明确不提供"总分"！这与 SPEC CPU 等传统基准测试不同。原因是：ML 系统通常是专用的，强制对所有任务加权平均会产生误导。这一设计决策反映了 ML 系统的根本特性——没有"万能"的 AI 芯片。

7 基准测试评估 (Benchmark Assessment)

原文翻译

10 月 11 日，我们将推理基准测试付诸实践。我们从 14 个组织收到了超过 600 个提交结果，涵盖所有三个类别（可用、预览和 RDO），跨封闭和开放分区。结果是向公众公开的最广泛的推理性能数据集，涵盖了一系列 ML 任务和场景、硬件架构和软件运行时。每个结果在被批准为有效的 MLPerf 结果之前都经过了广泛的审查。审查后，我们确认 595 个结果为有效。在本节中，我们根据推理基准测试目标评估封闭分区的结果。

7.1 任务覆盖

原文翻译

因为我们允许提交者选择任何任务来评估其系统性能，结果在任务中的分布可以揭示这些任务是否对 ML 系统供应商感兴趣。我们分析了提交结果以确定总体覆盖率。图 5 显示了封闭分区中任务和模型的分类。虽然最受欢迎的模型不出意料是 ResNet-50 v1.5，但它的受欢迎程度仅是最不受欢迎的模型 GNMT 的不到三倍。这种较小的差距和其他方面均匀的分布表明我们选择了具有代表性的任务集。

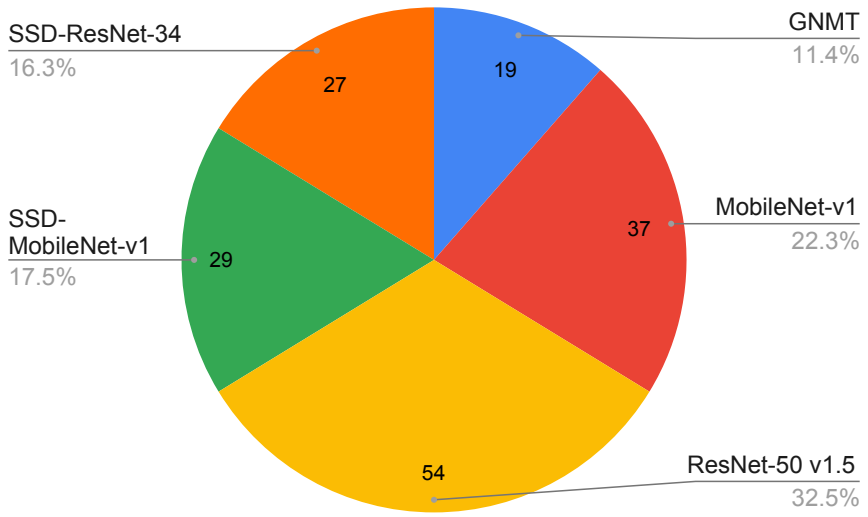

Figure 5: 封闭分区的结果。分布表明我们为基准测试的初始版本选择了具有代表性的工作负载。

图 5 详细解读

这是一个饼图，展示了封闭分区中各模型的提交数量分布：

- **最受欢迎**：ResNet-50 v1.5——作为"Hello World"级别的模型，几乎所有参与者都会提交
- **最不受欢迎**：GNMT——翻译任务，但也不是没人做
- **关键发现**：最热门和最冷门之间只差不到 3 倍，说明任务选择是均衡的

- 目标检测的两个模型（SSD-MobileNet 和 SSD-ResNet-34）提交数量相近，说明重量级和轻量级模型都有市场

这张图支持了论文的论点：MLPerf 选择的任务集是"具有代表性的"，而非偏向某一种任务。

Table 6: 模型和场景的高覆盖率。

	单流	多流	服务器	离线
GNMT	2	0	6	11
MobileNet-v1	18	3	5	11
ResNet-50 v1.5	19	5	10	20
SSD-MobileNet-v1	8	3	5	13
SSD-ResNet-34	4	4	7	12
总计	**51**	**15**	**33**	**67**

表 6 解读

这张表展示了封闭分区中各模型在各场景下的提交数量：

- **单流和离线最受欢迎**（分别 51 和 67 个提交）——这两个场景最容易优化和运行
- **多流和服务器较少**（分别 15 和 33）——有 QoS 要求且运行时间长
- **GNMT 多流为 0**——唯一没有任何提交的组合！可能因为固定到达间隔对翻译任务不太现实
- **ResNet-50 单流最多**（19个）——说明这是最"通用"的基准测试

7.2 场景使用

原文翻译

我们旨在在现实用例中评估系统——这是 LoadGen 和场景的主要动机。为此，表 6 显示了各种任务和场景组合中结果的分布。在所有任务中，单流和离线场景使用最广泛，也最容易优化和运行。服务器和多流更复杂，由于 QoS 要求和更多的查询，运行时间更长。GNMT 没有多流提交，可能因为固定到达间隔在机器翻译中不太现实。因此，它是唯一没有提交的模型和场景组合。
现实的 MLPerf Inference 场景是新颖的，说明了架构师面临但研究经常忽略的许多重要且复杂的性能考虑。图 6 表明，由于延迟约束和随之而来的次优批处理，所有系统在服务器场景中的吞吐量都低于离线场景。优化延迟是一项具有挑战性且被低估的任务。

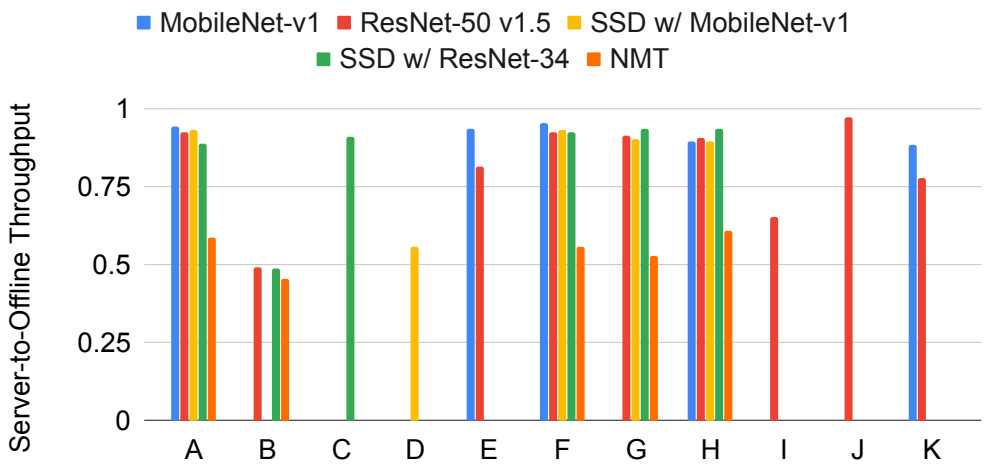

Figure 6: 从 11 个在封闭分区中任意选择的系统看服务器场景（有延迟约束）的吞吐量下降。每个模型的服务器场景性能归一化到同一模型在离线场景中的性能。分数 1 对应于一个模型在离线和服务器场景中提供相同的吞吐量。一些系统（包括 C、D、I 和 J）缺少某些模型的结果，因为参与者不需要为所有模型提交结果。

图 6 详细解读

这张图揭示了一个非常重要的洞察——**延迟约束对吞吐量的影响因系统和模型而异**：
横轴：11个不同的系统（A到K）。**纵轴**：归一化性能（1 = 服务器和离线性能相同）。
关键发现：

- **所有系统的服务器性能都低于离线**（所有柱子都 ≤ 1）
- **系统 B** 对所有模型损失 50% 以上的吞吐量——可能批处理优化不够好
- **系统 A** 对视觉模型只损失 ~10%，但对 NMT 损失 ~40%——视觉优化好于语言
- **NMT（翻译）**在所有系统上损失最大（39--55%）——因为输入长度可变、网络结构更复杂
- **MobileNet** 平均损失不到 10%——轻量级模型受延迟约束影响最小

核心结论：仅看离线/无约束吞吐量来比较系统是不充分的！在实际部署中延迟约束至关重要。

原文翻译

然而，并非所有系统同样地处理延迟约束。例如，系统 B 对所有三个模型损失约 50% 或更多的吞吐量，而系统 A 对 NMT 损失多达 40%，但对视觉模型仅约 10%。吞吐量下降差异可能是硬件架构针对低批大小进行了优化，或者推理引擎和软件栈中更有效的动态批处理的结果——或者更可能是两者的组合。即使数据有限，一个明确的含义是：具有无约束延迟的性能比较对于受延迟约束的场景没有什么参考价值。因此，性能分析理想情况下应同时包含两者。
此外，延迟约束的性能影响随网络类型而变化。在有 NMT 结果的全部五个系统中，服务器场景的吞吐量减少为 39--55%。相比之下，ResNet-50 v1.5 的吞吐量减少从 3% 到 35% 不等，平均约 20%，MobileNet-v1 的平均值低于 10%。巨大的吞吐量减少差异可能反映了 NMT 可变文本输入、更显著的软件栈优化以及 NMT 更复杂的网络架构的某种组合。来自这些数据的第二个教训是：延迟约束对不同模型的影响推断性很差。即使在分

类模型之间，ResNet-50 v1.5 的平均性能损失大约是 MobileNet-v1 的两倍。

7.3 处理器类型和软件框架

原文翻译

各种平台都可以使用 ML 解决方案，从完全通用的 CPU 到可编程 GPU 和 DSP、FPGA 以及固定功能加速器。我们的结果反映了这种多样性。图 7 表明 MLPerf Inference 提交覆盖了大多数硬件类别，表明我们的 v0.5 基准测试套件和方法可以评估任何处理器架构。

许多 ML 软件框架伴随着各种处理器类型。表 7 显示了基准测试硬件平台的框架。ML 软件在释放硬件性能方面起着至关重要的作用。一些运行时被设计为与某些硬件类型配合使用以充分利用其能力；使用没有相应框架的硬件可能仍然成功，但性能可能达不到硬件的潜力。表格显示 CPU 具有最多的框架多样性，TensorFlow 具有最多的架构多样性。

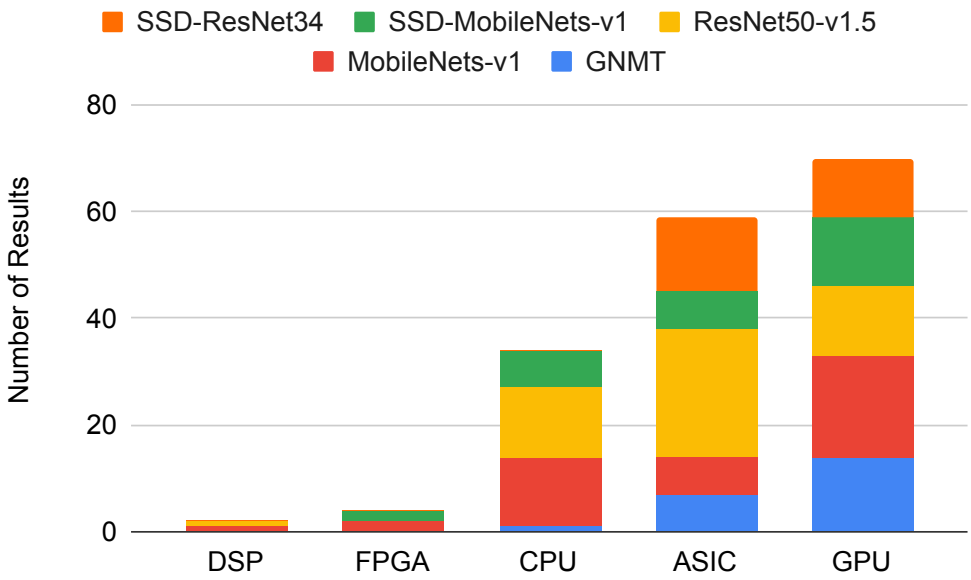

Figure 7: 封闭分区的结果。它们覆盖了几乎所有类型的处理器架构——CPU、GPU、DSP、FPGA 和 ASIC。

图 7 详细解读

这是一个堆叠柱状图，展示了不同处理器架构在各任务/场景组合中的分布：

- **GPU**（通常是 NVIDIA）占主导地位——特别是在需要高吞吐量的离线和服务器场景
- **CPU**（Intel 为主）在单流场景中有较多提交
- **ASIC**（如 Hailo、Habana、阿里含光）开始崭露头角
- **DSP**（Qualcomm）和 **FPGA**（Xilinx、FuriosaAI）也有参与
- 这证明了 MLPerf 的架构中立性——真正做到了"来者不拒"

Table 7: 框架与硬件架构的对应关系。

	ASIC	CPU	DSP	FPGA	GPU
Arm NN		X			X
FuriosaAI				X	
Hailo SDK	X				
HanGuang AI	X				
ONNX		X			
OpenVino		X			
PyTorch		X			
SNPE			X		
Synapse	X				
TensorFlow	X	X			X
TensorFlow Lite		X			
TensorRT					X

表 7 解读

这张表揭示了 ML 软件生态系统的碎片化：

- **CPU 框架最多**：Arm NN、ONNX、OpenVino、PyTorch、TensorFlow、TF Lite——反映了 CPU 的通用性
- **TensorFlow 覆盖最广**：支持 ASIC、CPU 和 GPU 三种架构
- **ASIC 各有专属框架**：Hailo SDK、HanGuang AI、Synapse——每家 ASIC 公司都有自己的工具链
- **GPU 主要用 TensorRT 和 TensorFlow**——NVIDIA 的生态优势明显
- 这种碎片化正是 MLPerf 存在的意义——需要一个统一的评估标准

7.4 系统多样性

原文翻译

提交覆盖了从移动和边缘设备到云计算的广泛功耗和性能范围。最小和最大推理系统之间的性能差距为四个数量级，约 10,000 倍。
图 8 显示了除 GNMT（MS）外所有任务和场景的结果，后者没有提交。在 MobileNet-v1 单流场景（SS）、ResNet-50 v1.5（SS）和 SSD-MobileNet-v1 离线（O）等情况下，系统表现出巨大的性能差异（100 倍）。因为这些模型有许多应用，针对它们的系统涵盖从低功耗嵌入式设备到高性能服务器的所有范围。GNMT 服务器（S）在系统之间表现出更少的性能变化。
广泛的性能范围意味着我们最初为 MLPerf Inference v0.5 选择的任务足够通用，可以代表许多用例和市场细分。广泛的系统阵列也表明我们的方法（LoadGen、度量指标等）

具有广泛的适用性。

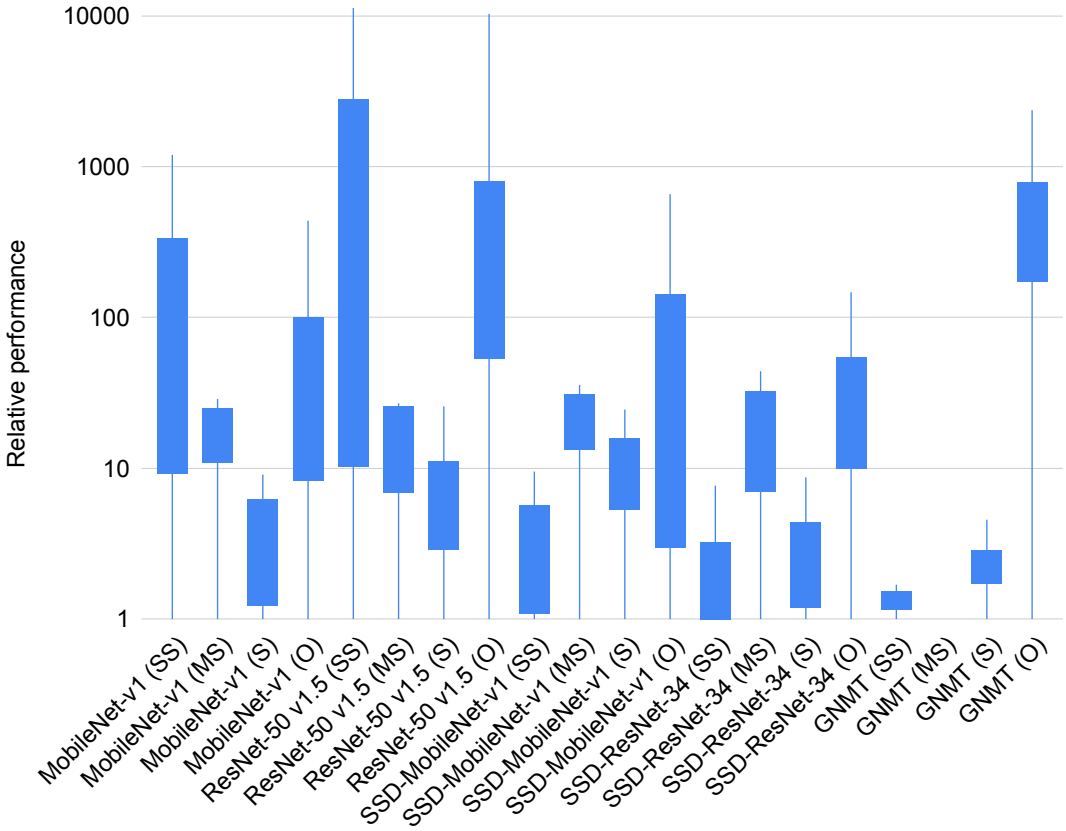

Figure 8: 不同模型在单流 (SS)、多流 (MS)、服务器 (S) 和离线 (O) 场景中的性能。分数相对于该特定场景中最慢系统的性能。

图 8 详细解读

这是一组小提琴图（violin plot），展示了每个任务+场景组合下所有提交系统的性能分布：
横轴：各种任务和场景的组合（如 MobileNet-v1 SS、ResNet-50 O 等）。
纵轴：归一化性能（对数刻度），相对于该组合中最慢系统的倍数。
关键发现：

- **性能差距惊人**：MobileNet-v1 SS 和 ResNet-50 SS 的差距达到 ~100倍甚至更多
- **小提琴形状**表示性能集中区域——宽的部分表示多个系统性能相近
- **GNMT Server**的分布最窄——因为只有少数大型系统能跑这个场景
- **离线场景**通常显示最大的性能跨度——从手机到数据中心全部参与

这张图有力地证明了 MLPerf 基准测试的广泛适用性。

7.5 开放分区

原文翻译

我们在限制较少的开放分区收到了 429 个结果。一些亮点包括使用 4 位量化来提升性能、探索各种模型（而非参考模型）来执行任务，以及在比封闭分区规则规定的更严格的延迟边界下实现高吞吐量。
我们还看到了推动移动芯片组性能极限的提交。通常，供应商一次使用一个加速器。我们看到了多个加速器并发工作以在多流场景中提供高吞吐量的实例——这在传统移动场景中很罕见。这些结果共同表明，开放分区正在鼓励行业推动系统极限。

8 经验教训 (Lessons Learned)

8.1 模型：广度 vs 用例深度

原文翻译

平衡应用的广度（如图像识别、目标检测和翻译）和模型（如 CNN 和 LSTM）与用例的深度（表 2）对行业很重要。因此我们实现了每个基准测试的 4 个版本，总共 20 个。有限的资源和对快速创新的需求使我们无法包含更多应用（如语音识别和推荐）和模型（如 Transformer**【引用说明：Vaswani2017 注意力机制——即将加入的新模型架构】**、BERT**【引用说明：Devlin2018 双向预训练Transformer——NLP新标准】**和 DLRM**【引用说明：Gupta2019、Naumov2019 Facebook的深度学习推荐模型——即将加入】**），但我们旨在尽快添加它们。

8.2 度量指标：延迟 vs 吞吐量

原文翻译

延迟和吞吐量密切相关，将它们一起考虑至关重要：我们使用延迟限制的吞吐量（表 3）。一个系统可以提供出色的吞吐量，但在出现延迟约束时表现不佳。例如，离线和服务器场景之间的区别在于后者施加了延迟约束并实现了非均匀到达率。结果是更低的吞吐量，因为大型输入批次变得更难形成。对于一些系统，服务器场景的延迟约束仅将性能降低 3%（相对于离线）；对于其他系统，损失要大得多（50%）。

8.3 数据集：公共 vs 私有

原文翻译

行业需要更大、更高质量的公共数据集用于 ML 系统基准测试。在调查了各种行业和学术场景后，我们发现 SSD-large 用例的输入图像大小范围很广，大约从 200x200 到 8 MP。我们选择了两种分辨率作为用例代理：小图像（0.09 MP，300x300）代表移动和一些数据中心应用，大图像（1.44 MP，1200x1200）代表机器人（包括自动驾驶汽车）和高端视频分析。然而实际上，SSD-large 使用了放大的（1200x1200）COCO 数据集（表 1），因为缺乏好的带有大图像的公共检测数据集。一些这样的数据集存在，但不太理想。ApolloScape**【引用说明：Wang2019 百度的自动驾驶数据集——有大图像但缺少边界框标注】**包含大图像但缺少边界框标注。Berkeley DeepDrive**【引用说明：Yu2018 Berkeley的多样驾驶视频数据库——分辨率较低(720p)】**的图像分辨率较低（720p）。我们需要更多的数据集生成器来解决这些问题。

8.4 性能：建模 vs 实测

原文翻译

虽然这是常见做法，但根据参数大小或运算计数（图 1）来表征网络的计算难度可能是过度简化。例如，在离线和服务器场景中有 10 个系统计算了 SSD-ResNet-34 和 SSD-MobileNet-v1 的性能。前者每张图像需要 175 倍的运算，但实际吞吐量仅低 50--60 倍。运算计数与观察到的性能之间一致的 3 倍差异显示了网络结构如何影响性能。

通俗解释

这个发现非常重要，对你做 CUDA 优化工作也很有启示：
SSD-ResNet-34 的 FLOPS 是 SSD-MobileNet-v1 的 175 倍，但实际吞吐量只慢 50--60 倍。为什么？

- **运算密度不同**：ResNet 的大卷积有更好的计算/访存比，GPU 利用率更高
- **MobileNet 的深度可分离卷积**计算密度低，虽然 FLOPS 少但内存访问多
- **结论**：FLOPS 不能直接预测实际性能。真正的瓶颈可能是内存带宽而非计算能力

这也是为什么需要实际基准测试而不是仅依赖理论计算量。

8.5 流程：审计和可审计性

原文翻译

因为提交者可以重新实现参考基准测试以最大化其系统能力，结果审查流程（第 6.2 节）对确保有效性和可重现性至关重要。我们在封闭分区的约 180 个结果中发现了约 40 个问题。我们最终只发布了其中 166 个结果。问题范围从未达到质量目标（表 1）、延迟边界（表 3）和查询要求（表 5）到不准确地解释规则。得益于 LoadGen 的准确度检查器（第 5.4 节）和提交检查脚本，我们自动识别了许多问题。检查器还减轻了负担，只需约三名工程师来梳理提交。总之，由于 ML 推理栈每个层级的选项多样性很复杂（图 2），我们发现审计和可审计性对确保结果完整性和可重现性是必要的。

9 AI/ML 基准测试的先前工作 (Prior Art in AI/ML Benchmarking)

原文翻译

MLPerf 努力吸收和建立在先前工作最佳方面的基础上，同时也包含社区输入。

原文翻译

AI Benchmark。AI Benchmark**【引用说明：Ignatov2019 AI Benchmark——首个移动推理基准套件，只关注Android智能手机】**可以说是第一个移动推理基准测试套件。其结果和排行榜专注于 Android 智能手机，仅测量延迟。该套件提供了一个总分，但未能明确指定质量目标。我们的目标是各种设备（我们的提交范围从 IoT 设备到智能手机和边缘/服务器规模系统）以及每个基准测试四个场景。
EEMBC MLMark。EEMBC MLMark**【引用说明：EEMBC MLMark——嵌入式推**

理设备的性能和准确度测量】测量嵌入式推理设备的性能和准确度。它也包含图像分类和目标检测任务，但缺乏用例场景。MLMark 在显式批大小下测量性能，而 MLPerf 允许提交者为不同场景选择最佳批大小。此外，前者不施加目标质量限制，而后者施加。

Fathom。Fathom**【引用说明：Adolf2016 Fathom——包含多种层类型的模型套件】**提供了一个包含多种层类型（如卷积、全连接和 RNN）的模型套件。然而，它专注于吞吐量而非准确度。与 Fathom 类似，我们包含了包含各种层的模型套件。与 Fathom 相比，MLPerf 提供了用于优化的 PyTorch 和 TensorFlow 参考实现，并引入了具有不同性能度量的各种推理场景。

AI Matrix。AI Matrix**【引用说明：Alibaba的AI Matrix——阿里巴巴的AI加速器基准测试】**是阿里巴巴的 AI 加速器基准测试。它使用微基准测试覆盖基本算子（如矩阵乘法和卷积），测量全连接和其他常见层的性能，包含紧密跟踪内部应用的完整模型，并提供合成基准测试以匹配真实工作负载的特征。MLPerf 有较少的模型集合，专注于使用 LoadGen 模拟场景。

DeepBench。DeepBench**【引用说明：Baidu2017 DeepBench——百度的深度学习内核级操作基准测试】**等微基准测试测量内核级操作（如 5,124×700×2,048 GEMM）的库实现，这些操作对生产模型的性能很重要。它们对高效开发有用，但未能解决完整模型的复杂性。

TBD（DNN训练基准测试）。TBD**【引用说明：Zhu2018 TBD——多伦多大学和微软研究院的DNN训练基准测试】**是多伦多大学和微软研究院的联合项目，专注于 ML 训练。它在三个框架（TensorFlow、MXNet 和 CNTK）中提供了广泛的 ML 模型，以及用于改进的强大工具链。它专注于评估 GPU 性能，只有一个完整模型（Deep Speech 2）覆盖推理。

DAWNBench。DAWNBench**【引用说明：Coleman2017 DAWNBench——第一个多参与者的深度学习端到端基准测试竞赛，启发了MLPerf】**是第一个测量深度学习系统端到端性能的多参与者基准测试竞赛。它允许跨模型架构、优化过程、软件框架和硬件平台的优化。DAWNBench 启发了 MLPerf，但我们提供了更多任务、模型和场景。

MLPerf vs 先前工作的关键区别

- **vs AI Benchmark**：MLPerf 覆盖更多设备类型（不仅是手机），有 4 种场景，有质量目标，无总分
- **vs EEMBC MLMark**：MLPerf 允许自选批大小，有质量目标，有用例场景
- **vs Fathom**：MLPerf 关注准确度+性能，有多种推理场景
- **vs AI Matrix**：MLPerf 使用 LoadGen 模拟现实场景
- **vs DeepBench**：MLPerf 测试完整模型而非单个内核
- **vs TBD**：MLPerf 专注推理而非训练
- **vs DAWNBench**：MLPerf 有更多任务、模型、场景，是工业界共识

10 结论 (Conclusion)

原文翻译

MLPerf Inference 的核心贡献是一个全面的框架，用于测量跨使用场景谱系的 ML 推理性能。我们简要总结推理基准测试的三个主要方面。

性能度量指标。为了对 AI 系统进行公平的同类比较，性能度量指标上的共识至关重要。我们精心制定了一系列这样的度量指标：延迟、延迟限制的吞吐量、吞吐量和每查询最大推理数——所有这些都受预定义的准确度目标和实现该目标的某种可能性的约束。

延迟或推理执行时间通常是系统和架构设计者采用的度量指标。相反，我们将延迟限制的吞吐量确定为工业用例中推理性能的度量，代表数据中心推理处理。虽然此度量对数据中心 CPU 是常见的，但我们为数据中心 ML 加速器引入了它。先前工作通常使用吞吐量或延迟；本文中的公式反映了更现实的部署约束。

准确度/性能权衡。ML 系统经常在准确度和性能之间权衡。先前工作在可接受的推理准确度降级方面差异很大。我们咨询了来自工业界和学术界的领域专家，为 MLPerf Inference 设置了准确度和可容忍的降级阈值，允许分布式测量和结果优化来调节准确度/性能权衡。这种方法标准化了 AI 系统的设计和评估，并使学术和工业研究能够使用 MLPerf Inference 工作负载的准确度要求来比较其努力与工业实现和已建立的准确度标准。

AI 推理加速器的评估。这项工作的一个重要贡献是识别和描述了 AI 推理加速器有用的度量指标和推理场景（服务器、单流、多流和离线）。加速器可能在一个类别中表现突出，而在另一个类别中表现不佳。这种退化归因于批处理（或缺乏批处理）等优化，这些优化取决于用例。MLPerf 在四个推理场景中的三个（服务器、多流和离线）跨五个网络引入了批处理，这些场景可以揭示 AI 系统开发和研究的额外优化。

ML 仍在发展。为了跟上变化的步伐，我们建立了维护 MLPerf 的流程**【引用说明：Mattson2020 MLPerf维护流程——描述如何保持基准测试与时俱进】**。我们正在为下一轮提交更新 ML 任务和模型，同时坚持本文描述的已建立的基本 ML 基准测试方法。尽管如此，我们还有很多要学习的。MLPerf 组织欢迎输入和贡献；请访问 https://mlperf.org/get-involved。

论文三大核心贡献总结

1. **性能度量标准化**：引入延迟限制吞吐量作为 ML 加速器的关键指标（而非简单延迟或吞吐量）
2. **准确度/性能权衡标准化**：设定行业共识的准确度阈值，使学术和工业研究可以公平比较
3. **推理场景系统化**：四种场景（服务器、单流、多流、离线）全面覆盖 AI 加速器的使用场景

致谢 (Acknowledgements)

原文翻译

MLPerf Inference 是来自多个组织的许多人的工作。在本节中，我们感谢所有帮助产生第一组结果或支持整体基准测试开发的人。

参与组织包括（按字母顺序）：Alibaba T-Head、Baidu、Cadence、Centaur Technology、Dell EMC、dividiti、Facebook、FuriosaAI、Google、Hailo、Intel、MediaTek、Microsoft、Myrtle、di Milano、Qualcomm、Samsung、Stanford University、Supermicro、University of Cal-

ifornia Berkeley / Google、Xilinx 以及多位非隶属贡献者。
超过 200 名工程师和研究人员直接参与了基准测试的开发、参考实现的构建和结果的提交。

附录：参考文献一览表

以下表格列出了论文中所有引用的文献及其在本文中的引用原因。

Table 8: 论文引用文献一览表

文献	主要内容	本文引用原因
He et al. 2016 (ResNet)	提出深度残差网络，解决深层网络训练问题	作为重量级图像分类的参考模型，同时说明ML应用范围
Goodfellow et al. 2014 (GAN)	提出生成对抗网络	说明计算机视觉应用的多样性
Liu et al. 2016 (SSD)	提出单次多框检测器	SSD-ResNet-34和SSD-MobileNet-v1的基础架构
Krizhevsky et al. 2012 (AlexNet)	ImageNet上深度CNN分类突破	说明ML在计算机视觉的里程碑
Vaswani et al. 2017 (Transformer)	提出注意力机制和Transformer架构	说明NLP应用；作为未来版本的候选模型
Devlin et al. 2018 (BERT)	预训练双向Transformer的语言理解	说明NLP应用；计划加入后续版本
Xu et al. 2018 (PointFusion)	3D目标检测的传感器融合	说明自动驾驶中的ML应用
Badrinarayanan et al. 2017 (SegNet)	编码器-解码器架构的语义分割	说明自动驾驶中的ML应用
Levine et al. 2018	机器人抓取的深度学习	说明ML在机器人领域的应用
Amodei & Hernandez 2018	AI训练所需算力的指数增长分析	说明训练是开发瓶颈和开销
Lee et al. 2019 (Facebook)	Facebook基础设施加速	提供推理规模数据（200万亿查询/日）
Kanter 2019	超算大会：AI芯片公司统计	超过100家公司做推理芯片
Tang 2019 (AI-Chip)	AI芯片公司列表	约20家公司做训练芯片
Bianco et al. 2018	DNN架构的基准分析	提供图1的数据——模型多样性
Howard et al. 2017 (MobileNets)	高效的移动端卷积网络	轻量级图像分类的参考模型
Hu et al. 2018 (SENet)	挤压激励网络	说明同准确度模型的计算差异
Xie et al. 2017 (ResNeXt)	聚合残差变换	说明同准确度模型的计算差异
Chollet 2017 (Xception)	深度可分离卷积	说明同准确度模型的计算差异
Deng et al. 2009 (ImageNet)	大规模层次图像数据库	图像分类任务的标准数据集
Lin et al. 2014 (COCO)	常见物体上下文数据集	目标检测任务的标准数据集
Wu et al. 2016 (GNMT)	Google的神经机器翻译系统	机器翻译任务的参考模型
WMT 2016	机器翻译大会	翻译任务的标准数据集
Papineni et al. 2002 (BLEU)	BLEU机器翻译自动评估方法	翻译质量度量
Post 2018 (SacreBLEU)	呼吁BLEU分数报告清晰度	推荐使用SacreBLEU确保可重现
Sandler et al. 2018 (MobileNetV2)	倒残差和线性瓶颈	评估但未选用的候选模型
Abadi et al. 2016 (TensorFlow)	Google大规模ML系统	参考实现的框架之一
Paszke et al. 2017 (PyTorch)	自动微分框架	参考实现的框架之一
Bai et al. 2019 (ONNX)	开放神经网络交换格式	参考实现的格式之一
Jia et al. 2014 (Caffe)	卷积架构快速特征嵌入	说明ML框架多样性
Tokui et al. 2015 (Chainer)	下一代开源DL框架	说明ML框架多样性
Seide et al. 2016 (CNTK)	微软开源深度学习工具包	说明ML框架多样性
Chollet et al. 2015 (Keras)	高层神经网络API	说明ML框架多样性
Chen et al. 2015 (MXNet)	灵活高效的分布式ML库	说明ML框架多样性

文献	主要内容	本文引用原因
Chetlur et al. 2014 (cuDNN)	GPU深度学习高效原语	说明优化库多样性
Intel MKL	数学核心库	说明优化库多样性
Khudia et al. 2018 (FBGEMM)	Facebook服务器端GEMM优化	说明优化库多样性
Apple CoreML	设备端ML框架	说明推理运行时多样性
Intel OpenVino	推理优化工具包	说明推理运行时多样性
NVIDIA TensorRT	可编程推理加速器	说明推理运行时多样性
Qualcomm SNPE	神经处理引擎SDK	说明推理运行时多样性
Lee et al. 2019 (TF-Lite)	移动GPU设备端推理	说明推理运行时多样性
Mattson et al. 2019 (MLPerf Training)	MLPerf训练基准测试	MLPerf推理基准的配套训练基准
MLPerf ResNet-50 v1.5	MLPerf规定的ResNet-50变体	确保ResNet-50实现一致性
Tamhane & Dunlop 2000	统计学与数据分析教材	提供查询数量计算的统计理论基础
Ignatov et al. 2019 (AI Benchmark)	首个移动推理基准	相关工作比较
EEMBC MLMark	嵌入式推理基准	相关工作比较
AIXPRT	封闭AI基准测试	相关工作比较（仅在原文注释中）
Adolf et al. 2016 (Fathom)	深度学习参考工作负载	相关工作比较
Coleman et al. 2017 (DAWNBench)	首个多参与者DL基准测试竞赛	相关工作比较；MLPerf的前身之一
Alibaba AI Matrix	AI加速器基准	相关工作比较
Baidu DeepBench	内核级DL操作基准	相关工作比较
Zhu et al. 2018 (TBD)	DNN训练基准	相关工作比较
Wang et al. 2019 (ApolloScape)	百度自动驾驶数据集	说明公共数据集不足的问题
Yu et al. 2018 (BDD100K)	Berkeley驾驶视频数据库	说明公共数据集分辨率不足
Gupta et al. 2019 (DLRM推理)	Facebook DNN推荐系统的架构影响	计划加入的新模型
Naumov et al. 2019 (DLRM训练)	深度学习推荐模型	计划加入的新模型
Mattson et al. 2020	MLPerf维护流程	说明基准测试的持续维护策略
Fursin et al. 2016 (CK)	集体知识工作流自动化	帮助开放分区中一个小团队扫描60+模型